Analysis of Iterative and Recursive Algorithms



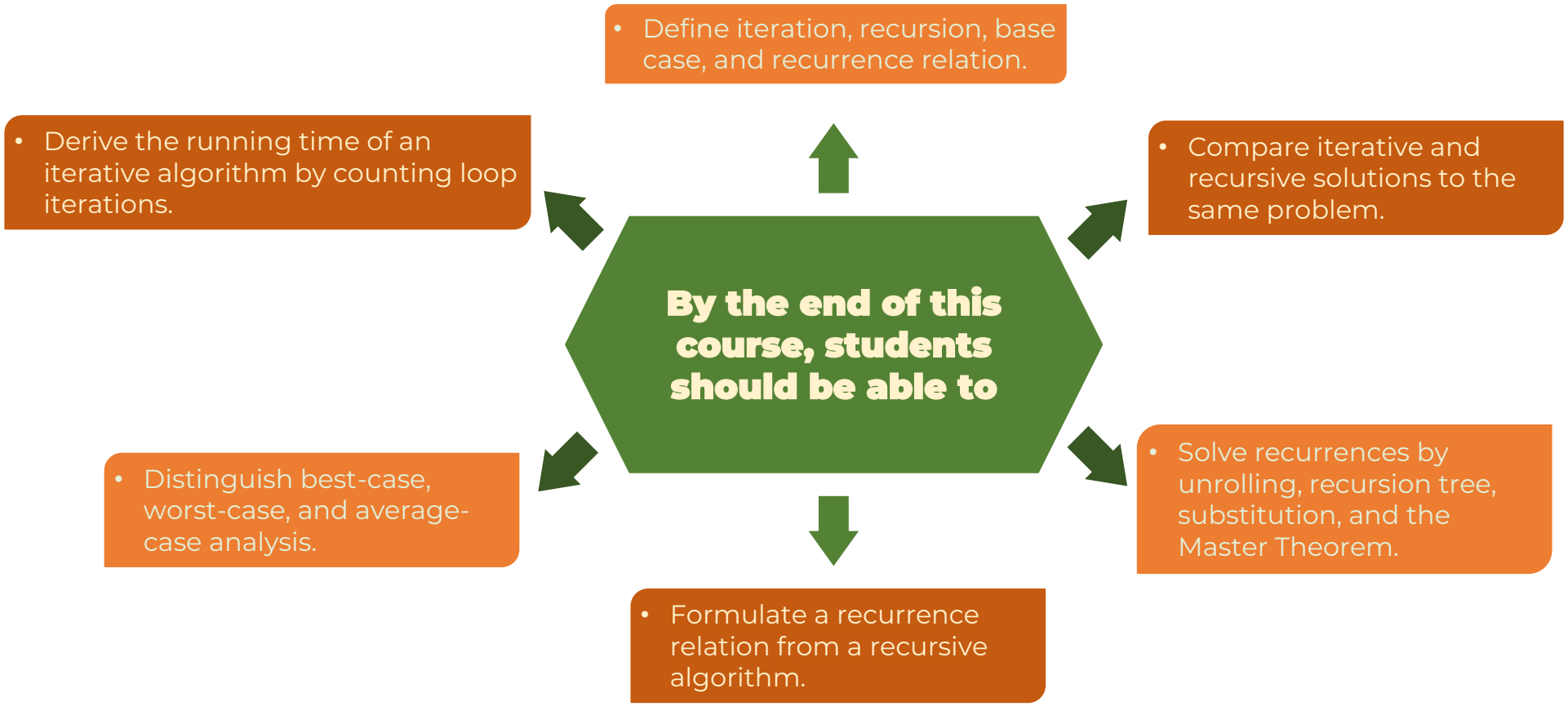
Lecture 3 — Analysis of Time Complexity

Abdelkrim Mebarki
Department of Computer Science

University of Science and Technology of Oran – Mohamed Boudiaf (USTOMB)
Academic Year : 2026 - 2027

https://sites.google.com/view/a-mebarki/

Learning Objectives



Roadmap of the Lecture

1. Loops	2. Cases	3. Recursion	4. Recurrences	5. Solving
Count iterations	Best, worst, average	Base case, recursive case	From code to equation	Unrolling, tree, substitution, Master Theorem

One question throughout

How many elementary operations are executed, as a function of the input size **n**?

- For **loops**, the answer comes from counting iterations.
- For **recursion**, it comes from an equation satisfied by the cost itself: a recurrence.
  - Solving such equations is the core of this lecture.

Key Definitions — Iteration & Loops

Term	Definition
Iterative algorithm	Repeats a block of instructions with loops (**for**, **while**) and never calls itself.
Loop body	The block of instructions repeated by the loop.
Control variable	The variable (such as **i**) that drives the loop, with its initial value, bound, and update step.
Iteration	One execution of the loop body.
Iteration count	Number of iterations, as a function of **n**: the central quantity of the analysis.
Total cost	Sum, over all iterations, of the cost of the body.

Consequence

- Once the iteration count is known,
- The cost follows: **Θ(iterations × cost of the body)** when the body cost is constant.

1 / ANALYZING ITERATIVE ALGORITHMS

The Method

The cost of an iterative algorithm is **the number of elementary operations**, as a function of **n**.

A four-step routine

1. **Choose** the input size **n**.
2. **Identify** the **elementary operations**.
3. **Count** their executions, structure by structure.
4. **Keep** the **dominant term** and express it with **Θ**.

Structure	Cost rule
Sequence **A ; B**	T(A) + T(B)
if … else …	T(test) + max(T(then), T(else))
for / **while** loop	**Sum** of the body cost over all iterations
Function call	Cost of the called body

- Only the iteration count changes from one loop pattern to another.
- Let us see the four patterns.

Simple & Nested Loops

```
SUM(A, n)
    s ← 0
    for i ← 0 to n-1 do
        s ← s + A[i]
    return s
```

```
ALL-PAIRS(A, n)
    for i ← 0 to n-1 do
        for j ← 0 to n-1 do
            process(A[i], A[j])
```

Simple loop

The body runs **n** times at constant cost **c**:

- **T(n) = c·n + c'**,
- hence **Θ(n)**.

Nested independent loops

The inner loop runs **n** times for each of the **n** outer iterations:

- multiply the counts, **n × n = n²**,
- hence $\Theta(n^2)$.
- With different bounds **n** and **m** the cost is **Θ(n·m)**.

Dependent & Logarithmic Loops

```
TRIANGLE(n)
    for i ← 0 to n-1 do
        for j ← 0 to i do
            process(i, j)
```

Dependent loops

The inner bound depends on the outer variable, so we sum instead of multiplying:

$$\sum_{i=0}^{n-1} (i+1)$$

- **Result :** $n(n+1)/2 \rightarrow \Theta(n^2)$

```
HALVINGS(n)
    i ← n
    while i ≥ 1 do
        process(i)
        i ← floor(i / 2)
```

Logarithmic loops

The variable is **divided** (or **multiplied**) by a constant at each iteration.

- For **n = 16**: i = **16**, **8**, **4**, **2**, **1**.
- Iterations = $\lfloor \log_2 n \rfloor + 1 \rightarrow \Theta(\log n)$

Loop Patterns at a Glance

Pattern	Iterations of the innermost body	Cost
Simple loop, step 1 (or constant step **k**)	**n** (or ≈ **n/k**)	**$\Theta(n)$**
Two nested independent loops	**$n \cdot n$**	**$\Theta(n^2)$**
Dependent loops (**$j \le i$**)	**$n(n+1)/2$**	**$\Theta(n^2)$**
Halving or doubling loop	**$\lfloor \log_2 n \rfloor + 1$**	**$\Theta(\log n)$**
Linear loop containing a halving loop	**$n \cdot \log n$**	**$\Theta(n \log n)$**

Question to ask before counting

- How many times does the innermost body execute?
- Yet some loops stop early (for example when a match is found). The cost then depends on the values in the input, not only on its size **n**. This leads to the next section.

2 / BEST-CASE, WORST-CASE AVERAGE-CASE ANALYSIS

2Best-Case, Worst-Case & Average-Case

For inputs of size n

Best case: **T_best(n) = min of T(I)**

Worst case: **T_worst(n) = max of T(I)**

Average case: **T_avg(n) = Σ P(I)·T(I)**

under a stated distribution

Linear search (comparisons with x)

Best → x at index 0 → 1 comparison → $\Theta(1)$.

Worst → x absent or last → n comparisons → $\Theta(n)$.

Average → x present, each position equally likely (probability 1/n): (1 + 2 + … + n)/n = (n+1)/2 → $\Theta(n)$.

- **Worst case** gives a guarantee;
- **Average case** requires an explicit model.

```
LINEAR-SEARCH(A, n, x)
    for i ← 0 to n-1 do
        if A[i] = x then
            return i
    return -1
```

When the Cases Diverge — Insertion Sort

```
INSERTION-SORT(A, n)
    for i ← 1 to n-1 do
        key ← A[i]
        j ← i - 1
        while j ≥ 0 and A[j] > key do
            A[j+1] ← A[j]
            j ← j - 1
        A[j+1] ← key
```

Case	Input	Comparisons	Cost
Best	Already sorted	n − 1	$\Theta(n)$
Worst	Reverse order	$n(n-1)/2$	$\Theta(n^2)$
Average	Random order	$\approx n^2/4$	$\Theta(n^2)$

The inner loop

- makes between 1 and i comparisons at step i:

  → the input order decides

Cases ≠ notations

- A case selects which inputs are considered.
- O, Ω, Θ describe how the resulting function grows.
- We can state Θ for the worst case, for the best case, or for the average case.

Recursion & Recurrences

Term	Definition
Recursive algorithm	Solves an instance by calling itself on smaller instances of the same problem.
Base case	Smallest instance, solved directly; it guarantees termination.
Recursive case	The part that reduces the instance and calls the algorithm again.
Recursion depth	Length of the longest chain of nested calls.
Recurrence relation	Equation defining **T(n)** from the values of **T** at smaller arguments, plus base value(s).
Closed form	Explicit formula for **T(n)**, with no **T** on the right-hand side.
Recursion tree	One node per call, each labeled with its own non-recursive cost.

Solving a recurrence means finding its closed form, or at least its Θ class.

3 / ANALYZING RECURSIVE FUNCTIONS

Analyzing Recursive Functions

Anatomy of a recursive function

A base case, a recursive case, and a parameter that progresses toward the base case.

```
FACTORIAL(n)
    if n = 0 then return 1
    return n × FACTORIAL(n-1)
```

```
FIB(n)
    if n ≤ 1 then return n
    return FIB(n-1) + FIB(n-2)
```

Cost = number of calls × work per call

- **FACTORIAL**: n + 1 calls, constant work each → $\Theta(n)$.
- **FIB**: every call spawns two calls. The number of calls is no longer obvious to count by hand.

→ We need a systematic tool: the recurrence relation.

Recurrence Relations

From Code to Equation

Recursive code → **Recurrence T(n)** → **Closed form** → **$\Theta(\cdot)$**

```
BIN-SEARCH-REC(A, low, high, x)
    if low > high then return -1
    mid ← floor((low + high) / 2)
    if A[mid] = x then return mid
    if A[mid] < x then
        return BIN-SEARCH-REC(A, mid+1, high, x)
    else
        return BIN-SEARCH-REC(A, low, mid-1, x)
```

Formulating T(n), with n = high − low + 1

- Base case: empty interval, $T(0) = c_0$.
- **Work outside the calls** → constant c.
- Recursive calls:

→ One call on at most n/2 elements (worst case).

$T(n) = T(\lfloor n/2 \rfloor) + c, \quad T(0) = c_0$

Formulating Recurrences

Two More Examples

```
HANOI(n, src, dst, tmp)
    if n = 0 then return
    HANOI(n-1, src, tmp, dst)
    move one disk from src to dst
    HANOI(n-1, tmp, dst, src)
```

HANOI (counting moves)

Two calls on size **n−1**, plus **one** move.

T(n) = 2T(n−1) + 1, T(0) = 0

```
SELECTION-SORT-REC(A, n)
    if n ≤ 1 then return
    m ← index of the maximum in A[0..n-1]
    swap A[m] and A[n-1]
    SELECTION-SORT-REC(A, n-1)
```

SELECTION-SORT-REC

One call on size **n−1**; the scan costs about **n**.

T(n) = T(n−1) + n, T(1) = c

Reading rule

- Number of calls → **coefficient**;
- Argument size → **index**;
- work outside the calls → **added term**.

4 / DIVIDE AND CONQUER

Recurrences — A New Shape:

Divide and Conquer

```
DIVIDE-AND-CONQUER(P, n)
    if n ≤ n₀ then
        return SOLVE-DIRECTLY(P)
    split P into a subproblems of size n/b
    for i ← 1 to a do
        Rᵢ ← DIVIDE-AND-CONQUER(Pᵢ, n/b)
    return COMBINE(R₁, …, Rₐ)
```

Algorithm	a	b	f(n)
Binary search	1	2	$\Theta(1)$
Merge Sort	2	2	$\Theta(n)$
Karatsuba multiplication	3	2	$\Theta(n)$
Block matrix multiplication	8	2	$\Theta(n^2)$

General recurrence

$T(n) = a \cdot T(n/b) + f(n)$, **$T(n) = \Theta(1)$** for **$n \le n_0$**

a ≥ 1: Number of recursive calls.

b > 1: Factor by which the size shrinks.

f(n): Cost of splitting and combining.

→ The algorithms themselves are studied later;

→ here we only need their recurrences.

→ Unrolling works on **T(n) = T(n/2) + c**

→ but becomes tedious for a = 2 or 3.

5 / SOLVING RECURRENCES

Four Methods

Method	Idea	Typical use
Unrolling	Expand T repeatedly, spot the pattern, sum	$T(n-1) + f(n)$, $T(n/2) + c$
Recursion tree	Draw the calls as a tree, add the cost level by level	Seeing the structure; guessing a bound
Substitution	Guess the solution, prove it by induction	Verifying a guess
Master Theorem	Read a, b, f(n) and apply a formula	$T(n) = aT(n/b) + f(n)$

Plan

We start with unrolling, then discover the tree, the induction proof, and finally the Master Theorem, which packages the tree analysis into a formula.

Unrolling

Expand the recurrence, spot the pattern after **k** steps, stop at the base case, then sum.

$T(n) = T(n-1) + c, \quad T(0) = c_0$

$$
\begin{aligned}
T(n) &= T(n-1) + c \\
&= T(n-2) + 2c \\
&= T(n-3) + 3c \\
&= \dots \\
&= T(n-k) + k \cdot c
\end{aligned}
$$

$k = n$: $T(n) = c_0 + c \cdot n$

$\Rightarrow \Theta(n)$

$T(n) = T(n-1) + n, \quad T(1) = 1$

$$
\begin{aligned}
T(n) &= T(n-1) + n \\
&= T(n-2) + (n-1) + n \\
&= T(n-3) + (n-2) + (n-1) + n \\
&= \dots \\
&= T(1) + 2 + 3 + \dots + n
\end{aligned}
$$

$= n(n+1)/2$

$\Rightarrow \Theta(n^2)$

Doubling and Halving

$T(n) = 2T(n-1) + 1, \quad T(0) = 0$

$$T(n) = 2T(n-1) + 1$$
$$= 4T(n-2) + 2 + 1$$
$$= 8T(n-3) + 4 + 2 + 1$$
$$= 2^k \cdot T(n-k) + (2^k - 1)$$

$k = n$: $T(n) = 2^n - 1 \Rightarrow \Theta(2^n)$

Checking the pattern by induction

Base: $T(0) = 0 = 2^0 - 1$.

Step: if $T(n-1) = 2^{n-1} - 1$, then $T(n) = 2(2^{n-1} - 1) + 1 = 2^n - 1$.

$T(n) = T(n/2) + c \quad (n = 2^k)$

$$T(n) = T(n/2) + c$$
$$= T(n/4) + 2c$$
$$= \ldots = T(n/2^k) + k \cdot c$$

$n/2^k = 1 \Rightarrow$ **$k = \log_2 n$**

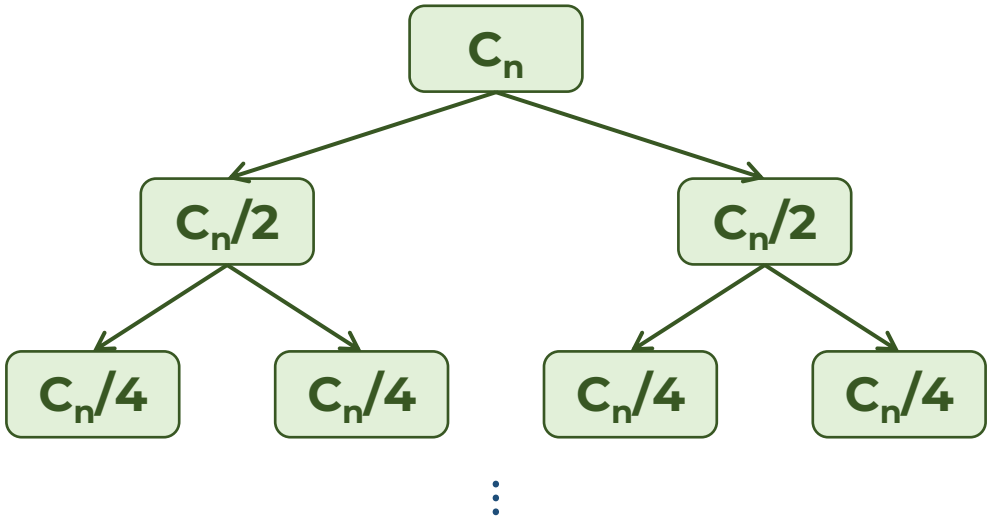
$T(n) = T(1) + c \cdot \log_2 n$

$\Rightarrow \Theta(\log n)$

Halving the size at each call mirrors the logarithmic loop seen earlier.

Recursion Tree

$T(n) = 2T(n/2) + C_n$



n leaves, each of cost c

Level	Nodes	Size	Cost of the level
0	1	n	C_n
1	2	n/2	C_n
i	2^i	$n/2^i$	C_n
...	...	...	...
$\log_2 n$	n	1	C_n

Each level costs $\mathbf{C_n}$, and there are $\mathbf{\log_2 n + 1}$ levels: $\mathbf{T(n) = \Theta(n \log n)}$.

What if the cost per level changes?

- If it grows toward the leaves, the leaves dominate.
- If it shrinks, the root dominates.
- The Master Theorem formalizes these three regimes.

Substitution

Guess, Then Prove by Induction

Procedure

1. **Guess** the form of the solution (from unrolling or a tree).
2. **Assume** it holds for smaller sizes (induction hypothesis).
3. **Prove :** Show it holds for **n**, choosing the constants.
4. **Check** the base case.

A wrong guess fails visibly

Assuming **T(n) ≤ c·n** gives **cn + n**, which exceeds **cn** for every **c**.

Claim: $T(n) \le c \cdot n \cdot \log_2 n$
for $T(n) = 2T(n/2) + n$, $T(1) = 1$,
n a power of 2, $n \ge 2$

Hypothesis: $T(n/2) \le c \cdot (n/2) \cdot \log_2(n/2)$

$T(n) \le 2 \cdot c \cdot (n/2) \cdot \log_2(n/2) + n$
$= c \cdot n \cdot (\log_2 n - 1) + n$
$= c \cdot n \cdot \log_2 n - c \cdot n + n$
$\le c \cdot n \cdot \log_2 n$ if $c \ge 1$

Base: $T(2) = 4 \le 2c$, so take $c = 2$.

Master Theorem — Statement

- Let **T(n) = a·T(n/b) + f(n)**, with **a ≥ 1, b > 1** constants, and **f(n)** asymptotically positive.
- The term **n/b** may be rounded up or down.
- Compare **f(n)** with the watershed function $n^{\log_b a}$.

Case	Condition on f(n)	Dominant part	Result	
1	$f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$	Leaves (recursion)	$\Theta(n^{\log_b a})$	The cost f(n) is asymptotically lower than the reference function.
2	$f(n) = \Theta(n^{\log_b a})$	All levels equal	$\Theta(n^{\log_b a} \cdot \log n)$	
3	$f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, and **a·f(n/b) ≤ c·f(n)** for some **c < 1**	Root (combine)	$\Theta(f(n))$	The cost f(n) is asymptotically higher than the reference function.

Shortcut when $f(n) = \Theta(n^d)$

$a < b^d \rightarrow \Theta(n^d)$ $\quad a = b^d \rightarrow \Theta(n^d \log n)$ $\quad a > b^d \rightarrow \Theta(n^{\log_b a})$

Master Theorem — Applying It

Four steps

1. Identify a, b, and f(n).
2. Compute $n^{\log_b a}$.
3. Compare **f(n)** with it: Polynomially smaller, equal, or polynomially larger.
4. Conclude, stating the case. In case 3, check **a·f(n/b) ≤ c·f(n)**.

Recurrence	a, b	$n^{\log_b a}$ vs f(n)	Case	Θ
T(n/2) + 1	1, 2	1 vs 1	2	Θ(log n)
2T(n/2) + n	2, 2	n vs n	2	Θ(n log n)
3T(n/2) + n	3, 2	$n^{1.58}$ vs n	1	$\Theta(n^{\log_2 3})$
8T(n/2) + n^2	8, 2	n^3 vs n^2	1	Θ(n^3)
4T(n/2) + n^2	4, 2	n^2 vs n^2	2	Θ(n^2 log n)
T(n/2) + n	1, 2	1 vs n	3	Θ(n)

Case 3 check for **T(n/2) + n**: **a·f(n/b) = n/2 ≤ ½·f(n)**.

Master Theorem — Limits & Extensions

When it does not apply

- Size decreases by subtraction: **T(n) = T(n−1) + n**. Use unrolling.
- **a** or **b** not constant: **T(n) = n·T(n/2) + 1**.
- **f(n)** falls in a gap between cases:

  T(n) = 2T(n/2) + n/log n.
- **Case 3** without the regularity condition.

Case 2 with logarithms

If **f(n) = $\Theta(n^{\log_b a} \cdot \log^k n)$, k ≥ 0**,

then $T(n) = \Theta(n^{\log_b a} \cdot \log^{(k+1)} n)$

→ Example: **$2T(n/2) + n \log n = \Theta(n \log^2 n)$**.

Subtractive recurrences

$T(n) = a \cdot T(n-b) + \Theta(n^k)$:

a	Result
a = 1	$\Theta(n^{(k+1)})$
a > 1	$\Theta(n^k \cdot a^{(n/b)})$

Examples: **$T(n-1) + n = \Theta(n^2)$**; **$2T(n-1) + 1 = \Theta(2^n)$.**

Bounding Recurrences

(FIB) and Summary of Results

$T(n) = T(n-1) + T(n-2) + c$ (FIB)

- Lower bound: $T(n-1) \geq T(n-2)$, so $T(n) \geq 2T(n-2)$, giving $T(n) = \Omega(2^{(n/2)})$.
- Upper bound: $T(n) \leq 2T(n-1) + c$,

  giving $T(n) = O(2^n)$.
- The cost is exponential. The exact growth is $\Theta(\varphi^n)$ with $\varphi \approx 1.618$.

Technique

Bound the recurrence between two easier ones.

Recurrence	Example	Θ
$T(n-1) + c$	FACTORIAL	$\Theta(n)$
$T(n-1) + n$	SELECTION-SORT-REC	$\Theta(n^2)$
$2T(n-1) + 1$	HANOI	$\Theta(2^n)$
$T(n/2) + c$	BIN-SEARCH-REC	$\Theta(\log n)$
$2T(n/2) + n$	MERGE-SORT	$\Theta(n \log n)$
$T(n-1) + T(n-2) + c$	FIB	$\Theta(\varphi^n)$

6. Iterative vs. Recursive Solutions

```
FIB-ITER(n)
    if n ≤ 1 then return n
    a ← 0 ; b ← 1
    for i ← 2 to n do
        c ← a + b
        a ← b ; b ← c
    return b
```

Criterion	Iterative	Recursive
Time, Fibonacci	$\Theta(n)$	$\Theta(\varphi^n)$
Time, factorial	$\Theta(n)$	$\Theta(n)$
Call stack	none	depth of recursion
Match with recursive structure	indirect	natural

Recursion is not slow by itself: FIB recomputes the same subproblems many times, which the loop avoids.

Analyze both versions, then choose according to clarity and measured cost. Stack memory is studied in Course 8.

What We Have Learned

- Iterative analysis means counting iterations: **simple $\Theta(n)$**, **nested independent $\Theta(n^2)$**, **dependent $\Theta(n^2)$**, **logarithmic $\Theta(\log n)$**.
- The cost may depend on the input itself: **best**, **worst**, and **average** cases are distinct questions.
- A recursive algorithm is analyzed through a **recurrence**: **base case**, **work per call**, **calls on smaller sizes**.
- Four methods solve recurrences: **unrolling**, **recursion tree**, **substitution**, and the **Master Theorem**.
- The **Master Theorem** compares **f(n)** with $n^{\log_b a}$ and gives three cases.
- Iterative and recursive solutions to the same problem can have very different costs.

Next course

Data Structures and Complexity: cost of searching, insertion, deletion, and access in trees, heaps, and hash tables.

References: CLRS, Introduction to Algorithms (Chapter 4); MIT 6.006; Stanford CS 161, Lecture 3.